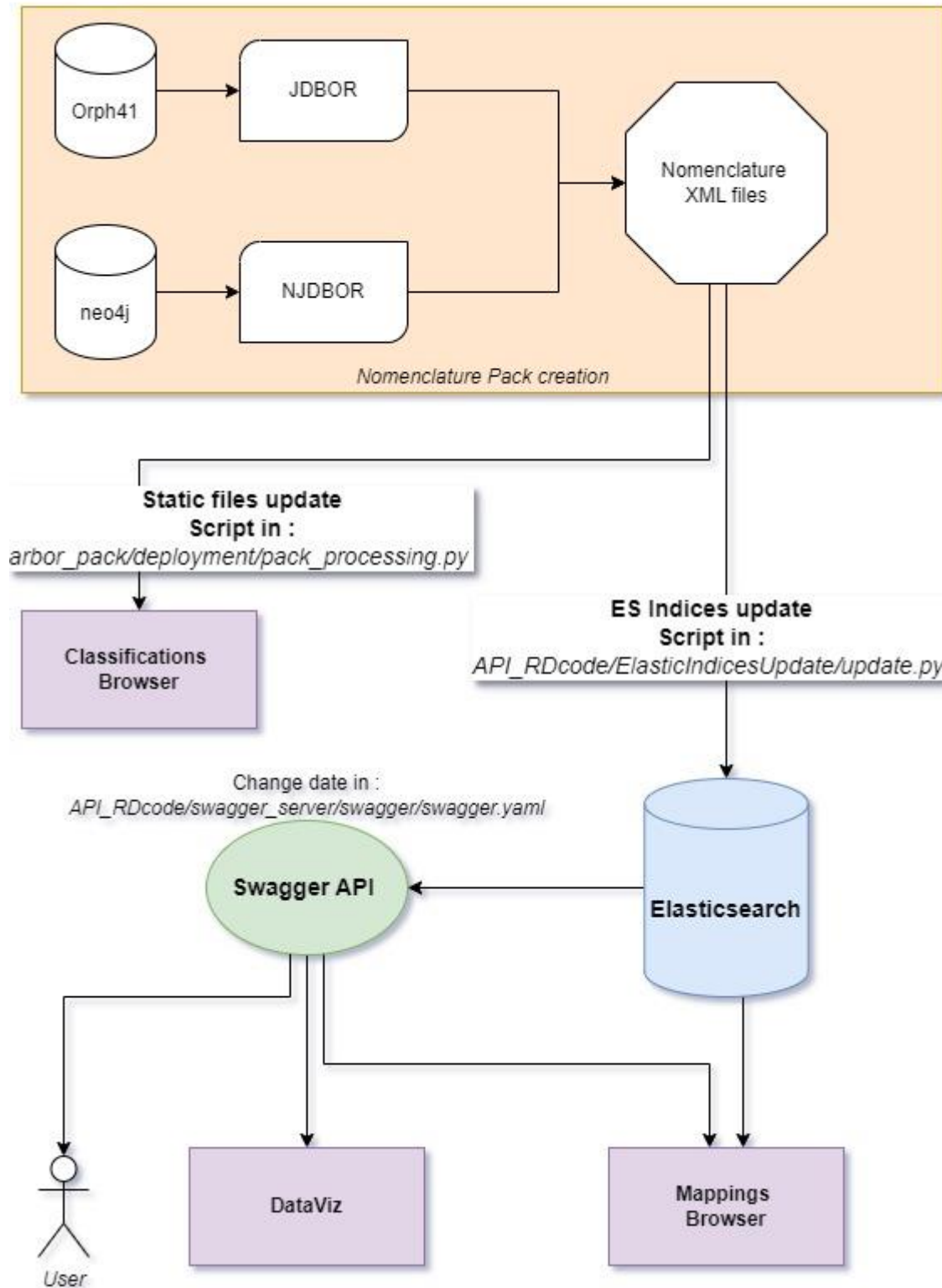


Transmission Orphacode

Stéphane Mabilie – 28/05/2026

Orphacode : Nomenclature Pack update



Le schéma ci-dessus représente l'écosystème Orphacode et ses méthodes de chargement/mise à jour depuis la création du pack Nomenclature jusqu'aux applications concernées.

Mise à jour des indices Elasticsearch

- Dans le git **API_RDcode**, au chemin **API_RDcode/ElasticIndicesUpdate/** se trouve le fichier *update.py* et sa version en Notebook *update.ipynb*. A l'exécution du script, les packs nomenclatures de toutes les langues sont chargés. L'architecture prévue dans le script (changeable au besoin) est la suivante : un dossier **NomenclaturePack25** contenant les packs de chaque langue, au même niveau que le dossier git (voir image).

```
smabille@ORPHANET16:~/Orphacode/tmp(0)$ tree -L 2
.
├── API_RDcode
│   ├── build
│   ├── ElasticIndicesUpdate
│   ├── gandi.yml
│   ├── media
│   ├── README.md
│   ├── requirements.txt
│   ├── setup.py.old
│   ├── swagger_server
│   ├── swagger_server.egg-info
│   ├── venv
│   └── wsgi.py
└── NomenclaturePack25
    ├── creation_date.txt
    ├── Orphanet_Nomenclature_Pack_CS
    ├── Orphanet_Nomenclature_Pack_DE
    ├── Orphanet_Nomenclature_Pack_EN
    ├── Orphanet_Nomenclature_Pack_ES
    ├── Orphanet_Nomenclature_Pack_FR
    ├── Orphanet_Nomenclature_Pack_IT
    ├── Orphanet_Nomenclature_Pack_NL
    ├── Orphanet_Nomenclature_Pack_PL
    ├── Orphanet_Nomenclature_Pack_PT
    └── ORPHA-SNOMEDCT_Mapping_File_production_2025.xlsx

18 directories, 7 files
```

Le script traite l'entière du pack Nomenclature, crée les indices Elasticsearch, puis les upload directement sur l'instance spécifiée. L'authentification à l'instance Elasticsearch se fait via l'utilisation d'un fichier **.varenv** contenant les variables utilisées.

NomenclaturePack25 = NomenclaturePackAPI

Mise à jour de l'API

- Une fois les indices Elasticsearch mis à jour, l'API se met à jour automatiquement. En effet, l'API ne fait que transmettre au serveur Elasticsearch les requêtes qui lui sont faites, les données sont donc à jour avec celle de l'instance. Cependant il peut être nécessaire de

modifier les endpoints ou les informations du Swagger. Pour cela, il faut modifier le fichier swagger : ***API_RDcode/swagger_server/swagger/swagger.yaml***, puis lancer la commande

```
java -jar build/tools/swagger-codegen-cli.jar generate -i  
swagger_server/swagger/swagger.yaml -l python-flask -o tmp_swagger/
```

Pour générer les controllers de l'API (***API_RDcode/swagger_server/controllers/***), dans lesquelles on peut éditer les fonctions correspondants au services décrits dans le fichier ***swagger.yaml***.

L'API peut être testée localement via le script ***API_RDcode/swagger_server/API_main.py*** avec wsgi.

Une fois l'API mise à jour, les applications **DataViz** et **Mappings browser** récupèrent automatiquement les données.

Mise à jour du Classifications Browser

- Le classifications browser ne dépendant ni de l'API ni du serveur Elasticsearch, il convient de générer les fichiers statiques utilisés par l'application à la main.
Le script ***arbor_pack/deployment/pack_processing.py*** utilise le pack Nomenclature anglais et le fichier mapping de SNOMED (***ORPHA-SNOMEDCT_Mapping_File_production_2025.xlsx***) pour générer un fichier statique ***arborData_000.json***, situé dans ***arbor_pack/static/processed_data***. A noter qu'il faut préciser dans le script le chemin utilisé pour les fichiers du pack (*base_path* = " ").
- Le Classifications Browser peut être testé localement avec la commande
`uwsgi --http 127.0.0.1:8081 --master -p 4 -w wsgi:application`